PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS – *CAMPUS* CONTAGEM
Instituto de Ciências Exatas e Informática – ICEI

Disciplina	Departamento	Período
Algoritmos e Estrutura de Dados	Sistemas de Informação	2º
Professores	**Email**	**Semestre**
Fábio Leandro Rodrigues Cordeiro	fabio@pucminas.br	2026/2

Estudo Dirigido – TAD Fila Flexível

Laboratório: implementar, rastrear, percorrer e medir uma fila com células encadeadas

Entrega: [data e hora de entrega] via Canvas **Valor:** [pontos]

1 Objetivos

Ao final do laboratório (aproximadamente 50 minutos) você deverá ser capaz de:

- implementar a Fila Flexível em C# com células encadeadas e nó cabeça;
- acompanhar, no papel, os ponteiros `primeiro` e `ultimo` a cada operação;
- percorrer a fila com um ponteiro auxiliar (pesquisar, maior, somar) e inverter a fila com uma pilha;
- encontrar defeitos em código encadeado e justificar por que o ponteiro `ultimo` existe.

Como trabalhar: atividade **individual**; consulte seus slides e anotações, mas escreva o próprio código. Use o arquivo `ED-FilaFlexivel-Esqueleto.cs`.

Regra do jogo: fora da classe `Fila`, só é permitido usar `inserir`, `remover`, `isVazia`, `tamanho` e `mostrar`. Dentro dela, `primeiro` e `ultimo` são os únicos ponteiros guardados; não crie contador de elementos como atributo.

2 Parte 1 – Implementando a Fila Flexível

A fila usa um **nó cabeça**: `primeiro` aponta para uma célula sem elemento útil, e o primeiro elemento da fila está em `primeiro.prox`. A fila está vazia quando `primeiro == ultimo`.

```
public class Celula
{
    public int elemento;
    public Celula prox;
    public Celula() : this(0) { }
    public Celula(int elemento) { /* TODO */ }
}

public class Fila
{
    private Celula primeiro, ultimo;
    public Fila() { /* TODO: nó cabeça */ }
    public bool isVazia() { /* TODO */ }
    public int tamanho() { /* TODO: percorra a fila contando */ }
    public void inserir(int x) { /* TODO: duas linhas */ }
    public int remover() { /* TODO */ }
    public void mostrar() { /* TODO: [3, 5, 7] */ }
}
```

Dica

`inserir` cabe em duas linhas: encadeie a nova célula depois de `ultimo` e depois avance `ultimo`. Em `remover`, a célula do elemento retirado vira o novo nó cabeça; guarde a antiga cabeça em uma variável temporária e desligue-a (`tmp.prox = null`).

Teste: execute a `Main` da Parte 1 e confira com a saída esperada que o professor mostrará no quadro. Qual resultado você espera para cada linha **antes** de executar?

3 Parte 2 – Rastreio no papel

Partindo de uma fila recém-criada, execute as operações abaixo e preencha a tabela. Em cada linha indique o conteúdo da fila (do primeiro ao último elemento), o valor devolvido por `remover` e se `primeiro == ultimo`.

#	Operação	Fila após a operação	Devolveu	`primeiro==ultimo`?
1	inserir(10)			
2	inserir(20)			
3	remover()			
4	inserir(30)			
5	remover()			
6	remover()			
7	remover()			
8	inserir(40)			

Pare e pense

(a) Na operação 6, para onde `primeiro` e `ultimo` apontam? (b) O que acontece na operação 7 e por quê? (c) Desenhe, para o estado após a operação 4, as células encadeadas com `primeiro` e `ultimo`; o que aconteceu com a célula que guardava o 10?

4 Parte 3 – Percorrendo a fila

Acrescente à classe `Fila` os métodos abaixo. Todos devem percorrer a fila com um ponteiro auxiliar `i` começando em `primeiro.prox`, sem remover elementos (exceto `inverter`).

- `bool pesquisar(int x)` – devolve se o valor está na fila;
- `int maior()` – devolve o maior elemento (exceção se a fila estiver vazia);
- `int somar()` – devolve a soma dos elementos;
- `void inverter()` – inverte a ordem da fila usando uma `Stack<int>` como apoio (apenas `inserir` e `remover` da própria fila).

Dica

Em `maior`, comece com o primeiro elemento real (`primeiro.prox.elemento`) e não com zero. Em `inverter`, retire tudo da fila empilhando, depois desempilhe inserindo de volta.

Execute a `Main` da Parte 3. Depois de `inverter`, insira o valor 1 e mostra a fila: por que isso é um bom teste para o ponteiro `ultimo`?

Desafio opcional – `removerUltimo`

Implemente `int removerUltimo()`, que retira o elemento do **fim** da fila. Qual é o custo? Por que não dá para fazer em tempo constante com uma única referência `prox`? (Descomente as linhas do desafio na `Main`.)

5 Parte 4 – Caça-erros

Cada trecho abaixo substitui o método correspondente da Parte 1. Para cada um, diga **o que acontece** e **como corrigir**, depois confirme no código.

```
// Versão A
public void inserir(int x) { ultimo.prox = new Celula(x); }

// Versão B
public void mostrar() {
    Console.Write("[");
    for (Celula i = primeiro; i != null; i = i.prox)
        Console.Write(i.elemento + (i.prox != null ? ", " : ""));
    Console.WriteLine("]");
}

// Versão C
public int tamanho() {
    int cont = 0;
    for (Celula i = primeiro.prox; i != ultimo; i = i.prox) cont++;
    return cont;
}
```

Dica: use as sequências `inserir(3); inserir(5); inserir(7);` em cada versão.

6 Parte 5 – Para que serve o `ultimo`?

A classe `FilaSemUltimo` não guarda o ponteiro `ultimo`. Implemente seu `inserir`: percorra a partir de `primeiro` até a última célula, incrementando `passos` a cada célula visitada, e encadeie a nova célula ali.

1. Execute a `Main` e anote o número de passos para $n = 1000$ e para $n = 2000$ inserções.
2. Dobrar n multiplicou os passos por quanto? Qual é a ordem de crescimento do custo total de n inserções? E o custo de uma inserção na fila com `ultimo`?
3. Compare com a Fila circular (vetor) da aula anterior: cite uma vantagem e uma desvantagem da fila flexível.

7 Entregáveis

Enviar, via Canvas, **um arquivo por aluno**: o código `.cs` completo (Partes 1, 3 e 5, e o desafio se feito) e um documento com a tabela da Parte 2 e as respostas das Partes 2 a 5, com seu nome e matrícula.